

MAT 103: Numerical Analysis I

Topic 4: Approximation of Functions by Polynomials

Dr. Anna Fome

Table of contents

1	Introduction	2
2	Lagrange Interpolation	2
2.1	The Main Idea	2
2.2	The Lagrange Basis Polynomials	3
2.3	The Lagrange Interpolating Polynomial	3
2.4	Solved Example 4.1 — Linear Lagrange Interpolation ($n = 1$) . .	3
2.5	Solved Example 4.2 — Quadratic Lagrange Interpolation ($n = 2$)	4
2.6	Solved Example 4.3 — Lagrange with Real Data	5
2.7	The Error Term in Lagrange Interpolation	6
2.7.1	Solved Example 4.4 — Error Bound	7
3	Finite Difference Interpolation Polynomials	7
3.1	Why Another Method?	7
3.2	The Difference Operator Δ	7
3.3	Building the Difference Table	8
3.3.1	Solved Example 4.5 — Building a Difference Table	8
3.4	Newton's Forward Difference Formula	9
3.4.1	Solved Example 4.6 — Newton Forward Interpolation . .	10
3.4.2	Solved Example 4.7 — Newton Forward on a Real Table .	11
3.5	Newton's Backward Difference Formula	12
3.5.1	Solved Example 4.8 — Newton Backward Interpolation .	13

“Polynomial approximation is the art of replacing something complicated with something simple — and knowing how much you have lost in doing so.”

1 Introduction

Suppose you are given a function $f(x)$ that is:

- defined only at a **few data points** (e.g. measurements from an experiment), or
- too **complicated** to differentiate or integrate analytically or
- a **black box** — you can evaluate it but have no formula.

In all these cases, we need a way to:

1. **Estimate** f at points where we have no data.
2. **Approximate** derivatives $f'(x)$, $f''(x)$.
3. **Approximate** definite integrals $\int_a^b f(x) dx$.

The key idea of this topic is: **replace** $f(x)$ by a polynomial $P(x)$ that passes through the known data points. Polynomials are easy to evaluate, differentiate, and integrate — so once we have $P(x)$, we can do all three tasks above.

This process is called **polynomial interpolation**.

i Definition

Polynomial interpolation is the construction of a polynomial $P(x)$ that passes exactly through a given set of data points:

$$(x_0, f_0), (x_1, f_1), \dots, (x_n, f_n)$$

where $f_i = f(x_i)$. The x_i are called **nodes** (or interpolation points).

By the end of this topic you should be able to:

- Build and apply the Lagrange interpolating polynomial.
- State and use the Lagrange error term.
- Build a forward and backward difference table.
- Apply Newton's Forward and Backward Difference formulas.
- Use an interpolating polynomial to approximate derivatives and integrals.
- Use MATLAB/MAPLE for interpolation.

2 Lagrange Interpolation

2.1 The Main Idea

Given $n + 1$ data points $(x_0, f_0), (x_1, f_1), \dots, (x_n, f_n)$, we want a polynomial $P_n(x)$ of degree **at most** n that satisfies:

$$P_n(x_i) = f_i \quad \text{for every } i = 0, 1, \dots, n$$

The clever trick in Lagrange interpolation is to build $P_n(x)$ from $n + 1$ simpler pieces called **Lagrange basis polynomials**, one for each node.

2.2 The Lagrange Basis Polynomials

For each node x_i , define:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j} = \frac{(x - x_0) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)}$$

The key property is:

$$L_i(x_j) = \begin{cases} 1 & \text{if } j = i \\ 0 & \text{if } j \neq i \end{cases}$$

In words: $L_i(x)$ equals **1 at its own node** and **0 at every other node**.

This is exactly what we need: it means when we add the pieces together, each f_i contributes only at x_i and nowhere else.

2.3 The Lagrange Interpolating Polynomial

$$P_n(x) = \sum_{i=0}^n f_i \cdot L_i(x) = f_0 L_0(x) + f_1 L_1(x) + \cdots + f_n L_n(x)$$

💡 Why Does This Work?

Check: at $x = x_k$, all $L_i(x_k) = 0$ except $L_k(x_k) = 1$. So $P_n(x_k) = f_k \cdot 1 = f_k$. The polynomial passes through every data point — exactly as required.

2.4 Solved Example 4.1 — Linear Lagrange Interpolation ($n = 1$)

Two data points: $x_0 = 1$, $f_0 = 2$ and $x_1 = 4$, $f_1 = 8$.

Estimate $f(2.5)$.

Step 1 — Build the basis polynomials:

$$L_0(x) = \frac{x - x_1}{x_0 - x_1} = \frac{x - 4}{1 - 4} = \frac{x - 4}{-3}$$

$$L_1(x) = \frac{x - x_0}{x_1 - x_0} = \frac{x - 1}{4 - 1} = \frac{x - 1}{3}$$

Step 2 — Evaluate at $x = 2.5$:

$$L_0(2.5) = \frac{2.5 - 4}{-3} = \frac{-1.5}{-3} = 0.5$$

$$L_1(2.5) = \frac{2.5 - 1}{3} = \frac{1.5}{3} = 0.5$$

Step 3 — Compute $P_1(2.5)$:

$$P_1(2.5) = f_0 \cdot L_0(2.5) + f_1 \cdot L_1(2.5) = 2(0.5) + 8(0.5) = 1 + 4 = 5$$

Result: $f(2.5) \approx 5$.

(Note: $L_0 + L_1 = 0.5 + 0.5 = 1$ — the basis polynomials always sum to 1.)

2.5 Solved Example 4.2 — Quadratic Lagrange Interpolation ($n = 2$)

Three data points:

x	1	3	5
$f(x)$	1	9	25

Estimate $f(2)$.

(Note: these values come from $f(x) = x^2$.)

Step 1 — Build the three basis polynomials:

$$L_0(x) = \frac{(x - 3)(x - 5)}{(1 - 3)(1 - 5)} = \frac{(x - 3)(x - 5)}{(-2)(-4)} = \frac{(x - 3)(x - 5)}{8}$$

$$L_1(x) = \frac{(x - 1)(x - 5)}{(3 - 1)(3 - 5)} = \frac{(x - 1)(x - 5)}{(2)(-2)} = \frac{(x - 1)(x - 5)}{-4}$$

$$L_2(x) = \frac{(x-1)(x-3)}{(5-1)(5-3)} = \frac{(x-1)(x-3)}{(4)(2)} = \frac{(x-1)(x-3)}{8}$$

Step 2 — Evaluate each L_i at $x = 2$:

$$L_0(2) = \frac{(2-3)(2-5)}{8} = \frac{(-1)(-3)}{8} = \frac{3}{8}$$

$$L_1(2) = \frac{(2-1)(2-5)}{-4} = \frac{(1)(-3)}{-4} = \frac{-3}{-4} = \frac{3}{4}$$

$$L_2(2) = \frac{(2-1)(2-3)}{8} = \frac{(1)(-1)}{8} = \frac{-1}{8}$$

Check: $L_0 + L_1 + L_2 = \frac{3}{8} + \frac{3}{4} - \frac{1}{8} = \frac{3+6-1}{8} = \frac{8}{8} = 1$

Step 3 — Compute $P_2(2)$:

$$P_2(2) = 1 \cdot \frac{3}{8} + 9 \cdot \frac{3}{4} + 25 \cdot \left(-\frac{1}{8}\right) = \frac{3}{8} + \frac{27}{4} - \frac{25}{8} = \frac{3+54-25}{8} = \frac{32}{8} = 4$$

Result: $P_2(2) = 4$.

True value: $f(2) = 2^2 = 4$ (exact, because $f(x) = x^2$ is a degree-2 polynomial and we used 3 nodes — so the interpolant reproduces f exactly).

2.6 Solved Example 4.3 — Lagrange with Real Data

The following table gives values of $f(x) = \ln(x)$:

x	2	4	6
$f(x) = \ln(x)$	0.6931	1.3863	1.7918

Estimate $\ln(5)$ using Lagrange interpolation.

(*True value:* $\ln(5) = 1.6094$)

Step 1 — Basis polynomials at $x = 5$:

$$L_0(5) = \frac{(5-4)(5-6)}{(2-4)(2-6)} = \frac{(1)(-1)}{(-2)(-4)} = \frac{-1}{8}$$

$$L_1(5) = \frac{(5-2)(5-6)}{(4-2)(4-6)} = \frac{(3)(-1)}{(2)(-2)} = \frac{-3}{-4} = \frac{3}{4}$$

$$L_2(5) = \frac{(5-2)(5-4)}{(6-2)(6-4)} = \frac{(3)(1)}{(4)(2)} = \frac{3}{8}$$

Check: $-\frac{1}{8} + \frac{3}{4} + \frac{3}{8} = \frac{-1+6+3}{8} = \frac{8}{8} = 1$

Step 2 — Interpolated value:

$$\begin{aligned} P_2(5) &= 0.6931 \left(-\frac{1}{8}\right) + 1.3863 \left(\frac{3}{4}\right) + 1.7918 \left(\frac{3}{8}\right) \\ &= -0.0866 + 1.0397 + 0.6719 = 1.6250 \end{aligned}$$

Result: $\ln(5) \approx 1.6250$

Error: $|1.6250 - 1.6094| = 0.0156$ — reasonable for such widely spaced nodes.

2.7 The Error Term in Lagrange Interpolation

Whenever we use $P_n(x)$ to approximate $f(x)$, there is an error. The **Lagrange error formula** tells us exactly how large that error can be.

i Lagrange Error Formula

If f has $n + 1$ continuous derivatives, the error at any point x is:

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

for some unknown ξ between the smallest and largest of $x, x_0, x_1, \dots, x_n$.

What does this tell us?

- The error depends on how large $f^{(n+1)}$ is — if f changes rapidly, the error is larger, else the error is small. $(n+1)!$ appears in the denominator help to reduce the error.
- Using **more nodes** (larger n) increases the degree and reduces the error, provided $f^{(n+1)}$ stays manageable.

Practical bound: Since ξ is unknown, we replace $f^{(n+1)}(\xi)$ with its maximum value M_{n+1} on the interval:

$$|f(x) - P_n(x)| \leq \frac{M_{n+1}}{(n+1)!} \left| \prod_{i=0}^n (x - x_i) \right|$$

2.7.1 Solved Example 4.4 — Error Bound

For Example 4.2 ($f(x) = x^2$, nodes at 1, 3, 5, query at $x = 2$):

$n = 2$, so we need $f^{(3)}(x)$. Since $f(x) = x^2$, $f^{(3)}(x) = 0$ everywhere.

$$\text{Error} = \frac{0}{3!} \times \dots = 0$$

The interpolation is **exact** — as expected, since $P_2(x)$ reproduces any polynomial of degree ≤ 2 without error.

For $f(x) = \ln(x)$ (Example 4.3), $f^{(3)}(x) = \frac{2}{x^3}$. On $[2, 6]$, the maximum is $\frac{2}{2^3} = 0.25$.

$$|E| \leq \frac{0.25}{6} |(5-2)(5-4)(5-6)| = \frac{0.25}{6} \times 3 = 0.125$$

Our actual error was 0.0156 — well within this bound

3 Finite Difference Interpolation Polynomials

3.1 Why Another Method?

Lagrange interpolation works well for any set of nodes $x_0, x_1, \dots, x_n$. But when the nodes are **equally spaced** — that is, $x_i = x_0 + i \cdot h$ for a fixed step size h — we can use a much more systematic approach called **finite difference interpolation**.

This approach:

- organises all the data in a neat **difference table** (built once, used many times),
- gives simple formulas for interpolation, differentiation, and integration,
- is historically very important and still widely used in numerical analysis.

3.2 The Difference Operator Δ

For equally spaced nodes with step size h , the **forward difference operator** Δ is:

$$\Delta f_i = f_{i+1} - f_i$$

This is simply the change in f from one node to the next. We can apply Δ repeatedly to get **higher-order differences**:

$$\Delta^2 f_i = \Delta(\Delta f_i) = \Delta f_{i+1} - \Delta f_i = f_{i+2} - 2f_{i+1} + f_i$$

$$\Delta^3 f_i = \Delta(\Delta^2 f_i) = \Delta^2 f_{i+1} - \Delta^2 f_i$$

In general: $\Delta^k f_i = \Delta^{k-1} f_{i+1} - \Delta^{k-1} f_i$

3.3 Building the Difference Table

The **forward difference table** organises all the differences in a triangular array:

x_i	f_i	Δf_i	$\Delta^2 f_i$	$\Delta^3 f_i$	$\Delta^4 f_i$
x_0	f_0	Δf_0	$\Delta^2 f_0$	$\Delta^3 f_0$	$\Delta^4 f_0$
x_1	f_1	Δf_1	$\Delta^2 f_1$	$\Delta^3 f_1$	
x_2	f_2	Δf_2	$\Delta^2 f_2$		
x_3	f_3	Δf_3			
x_4	f_4				

Rule: every entry = (entry below it) – (entry beside it on the left). Equivalently: every entry = (entry below-left) – (entry beside-left).

💡 How to Build the Table — One Step at a Time

Work **column by column**, left to right. Each new column is obtained by subtracting consecutive entries of the previous column. The top entry of each column is what we call $\Delta^k f_0$.

3.3.1 Solved Example 4.5 — Building a Difference Table

Build the forward difference table for:

x	1	2	3	4
$f(x) = x^3$	1	8	27	64

Here $h = 1$, $x_0 = 1$.

Column 1 — First differences Δf_i :

$$\begin{aligned}\Delta f_0 &= f_1 - f_0 = 8 - 1 = 7 \\ \Delta f_1 &= f_2 - f_1 = 27 - 8 = 19 \\ \Delta f_2 &= f_3 - f_2 = 64 - 27 = 37\end{aligned}$$

Column 2 — Second differences $\Delta^2 f_i$:

$$\begin{aligned}\Delta^2 f_0 &= \Delta f_1 - \Delta f_0 = 19 - 7 = 12 \\ \Delta^2 f_1 &= \Delta f_2 - \Delta f_1 = 37 - 19 = 18\end{aligned}$$

Column 3 — Third differences $\Delta^3 f_i$:

$$\Delta^3 f_0 = \Delta^2 f_1 - \Delta^2 f_0 = 18 - 12 = 6$$

Complete table:

x_i	f_i	Δf_i	$\Delta^2 f_i$	$\Delta^3 f_i$
1	1	7	12	6
2	8	19	18	
3	27	37		
4	64			

The **bolded top diagonal** — $f_0 = 1$, $\Delta f_0 = 7$, $\Delta^2 f_0 = 12$, $\Delta^3 f_0 = 6$ — is what we will use in the Newton Forward formula.

i Useful Pattern

For a polynomial of degree k , the k -th differences are **constant** and all higher differences are **zero**. Since $f(x) = x^3$ is degree 3, $\Delta^3 f = 6$ (constant) and $\Delta^4 f = 0$ everywhere.

3.4 Newton's Forward Difference Formula

When we want to interpolate **near the beginning of the table** (i.e., x is close to x_0), we use Newton's Forward Difference formula.

Define the dimensionless variable:

$$s = \frac{x - x_0}{h}$$

So $s = 0$ at x_0 , $s = 1$ at x_1 , $s = 2$ at x_2 , and so on. For a point **between** x_0 and x_1 , we have $0 < s < 1$.

The formula is:

$$P_n(x) = f_0 + s \Delta f_0 + \frac{s(s-1)}{2!} \Delta^2 f_0 + \frac{s(s-1)(s-2)}{3!} \Delta^3 f_0 + \dots$$

Or more compactly, using binomial-style notation:

$$P_n(x) = \sum_{k=0}^n \binom{s}{k} \Delta^k f_0, \quad \text{where } \binom{s}{k} = \frac{s(s-1)(s-2) \cdots (s-k+1)}{k!}$$

Only the top row of the difference table is used: $f_0, \Delta f_0, \Delta^2 f_0, \Delta^3 f_0, \dots$

3.4.1 Solved Example 4.6 — Newton Forward Interpolation

Using the table from Example 4.5, estimate $f(1.5)$.

Here $x_0 = 1$, $h = 1$, $x = 1.5$.

$$s = \frac{1.5 - 1}{1} = 0.5$$

From the top row: $f_0 = 1$, $\Delta f_0 = 7$, $\Delta^2 f_0 = 12$, $\Delta^3 f_0 = 6$.

$$P_3(1.5) = f_0 + s \Delta f_0 + \frac{s(s-1)}{2} \Delta^2 f_0 + \frac{s(s-1)(s-2)}{6} \Delta^3 f_0$$

Compute each term:

Term	Calculation	Value
f_0	1	1
$s \Delta f_0$	0.5×7	3.5
$\frac{s(s-1)}{2} \Delta^2 f_0$	$\frac{0.5 \times (-0.5)}{2} \times 12 =$ $\frac{-0.25}{2} \times 12$	-1.5
$\frac{s(s-1)(s-2)}{6} \Delta^3 f_0$	$\frac{0.5 \times (-0.5) \times (-1.5)}{6} \times$ $6 = \frac{0.375}{6} \times 6$	0.375

$$P_3(1.5) = 1 + 3.5 - 1.5 + 0.375 = 3.375$$

True value: $(1.5)^3 = 3.375$ (exact, because $f(x) = x^3$ is degree 3 and we used 4 nodes)

3.4.2 Solved Example 4.7 — Newton Forward on a Real Table

The following table gives values of e^x :

x	0.0	0.1	0.2	0.3
e^x	1.000000	1.105171	1.221403	1.349859

Estimate $e^{0.15}$.

(*True value:* $e^{0.15} = 1.161834$)

Step 1 — Build the difference table ($h = 0.1, x_0 = 0.0$):

x	f	Δf	$\Delta^2 f$	$\Delta^3 f$
0.0	1.000000	0.105171	0.011061	0.001163
0.1	1.105171	0.116232	0.012224	
0.2	1.221403	0.128456		
0.3	1.349859			

Step 2 — Compute s :

$$s = \frac{0.15 - 0.0}{0.1} = 1.5$$

Step 3 — Apply the formula (using top row values in bold):

Term	Calculation	Value
f_0	1.000000	1.000000
$s \Delta f_0$	1.5×0.105171	0.157757
$\frac{s(s-1)}{2} \Delta^2 f_0$	$\frac{1.5 \times 0.5}{2} \times 0.011061 =$ 0.375×0.011061	0.004148
$\frac{s(s-1)(s-2)}{6} \Delta^3 f_0$	$\frac{1.5 \times 0.5 \times (-0.5)}{6} \times$ $0.001163 =$ -0.0625×0.001163	-0.000073

$$P_3(0.15) = 1.000000 + 0.157757 + 0.004148 - 0.000073 = 1.161832$$

True value: $e^{0.15} = 1.161834$ — error of only 0.000002!

3.5 Newton's Backward Difference Formula

When we want to interpolate **near the end of the table** (i.e., x is close to x_n , the last node), we use the **Backward Difference formula**.

First define the **backward difference operator** ∇ :

$$\nabla f_i = f_i - f_{i-1}$$

$$\nabla^2 f_i = \nabla f_i - \nabla f_{i-1} = f_i - 2f_{i-1} + f_{i-2}$$

Now define s measuring distance **backwards** from the last node x_n :

$$s = \frac{x - x_n}{h} \quad (\text{note: } s \leq 0 \text{ for } x \leq x_n)$$

The **Newton Backward Difference Formula** is:

$$P_n(x) = f_n + s \nabla f_n + \frac{s(s+1)}{2!} \nabla^2 f_n + \frac{s(s+1)(s+2)}{3!} \nabla^3 f_n + \dots$$

Only the bottom row (last diagonal) of the difference table is used:
 $f_n, \nabla f_n, \nabla^2 f_n, \nabla^3 f_n, \dots$

The backward differences can be read from the **bottom-right diagonal** of the same difference table built in the forward direction:

x_i	f_i	Δf_i	$\Delta^2 f_i$	$\Delta^3 f_i$
x_0	f_0	Δf_0	$\Delta^2 f_0$	${}^3\mathbf{f}_0 = {}^3\mathbf{f}_3$
x_1	f_1	Δf_1	${}^2\mathbf{f}_1 = {}^2\mathbf{f}_3$	
x_2	f_2	$\mathbf{f}_2 = \mathbf{f}_3$		
x_3	$\mathbf{f}_3$			

So $\nabla f_n = \Delta f_{n-1}$, $\nabla^2 f_n = \Delta^2 f_{n-2}$, etc.

3.5.1 Solved Example 4.8 — Newton Backward Interpolation

Using the e^x table from Example 4.7, estimate $e^{0.25}$.

(True value: $e^{0.25} = 1.284025$)

$x_n = 0.3$, $h = 0.1$, $x = 0.25$:

$$s = \frac{0.25 - 0.3}{0.1} = -0.5$$

Bottom-right diagonal (backward differences at $x_n = 0.3$):

$$f_n = 1.349859, \quad \nabla f_n = \Delta f_2 = 0.128456, \quad \nabla^2 f_n = \Delta^2 f_1 = 0.012224, \quad \nabla^3 f_n = \Delta^3 f_0 = 0.001163$$

Apply the backward formula:

Term	Calculation	Value
f_n	1.349859	1.349859
$s \nabla f_n$	$(-0.5)(0.128456)$	-0.064228
$\frac{s(s+1)}{2} \nabla^2 f_n$	$\frac{(-0.5)(0.5)}{2} \times 0.012224 = -0.125 \times 0.012224$	-0.001528
$\frac{s(s+1)(s+2)}{6} \nabla^3 f_n$	$\frac{(-0.5)(0.5)(1.5)}{6} \times 0.001163 = -0.0625 \times 0.001163$	-0.000073

$$P_3(0.25) = 1.349859 - 0.064228 - 0.001528 - 0.000073 = 1.284030$$

True value: $e^{0.25} = 1.284025$ — error of only 0.000005